XAI-Driven Decentralized Infrastructure Monitoring and Safety Navigation System

Shreevathsa K[1], Shridattha M H[2], Vipul S[3], Gokul A S[4], Anirudh K[5], Dr. Anitha Sandeep[6]
[1,2,3]*Department of Electronics and Communication Engineering,*
[4]*Department of Electronics and Instrumentation Engineering,*
[5]*Department of Electronics and Telecommunication Engineering,*
[6]*Department of Computer Science and Engineering*
RV College of Engineering, Bengaluru 560059, India
[1]*shreevathsak.ec23@rvce.edu.in*, [2]*shridatthamh.ec23@rvce.edu.in*, [3]*vipuls.ec23@rvce.edu.in*,
[4]*gokuladithyas.ei23@rvce.edu.in*, [5]*anirudhk.et23@rvce.edu.in*, [6]*anithasandeep@rvce.edu.in*

***Abstract*—This paper presents an Explainable Artificial Intelligence (XAI)–enabled, edge-based framework for infrastructure safety monitoring, indoor localization, and emergency evacuation guidance. The system integrates multi-modal environmental sensing—including fire, gas concentration, and temperature—with Bluetooth Low Energy (BLE)-based indoor positioning to enable real-time evacuation routing and navigation.**

An RSSI-weighted multilateration method optimized via iterative gradient descent improves localization robustness, while a hazard-weighted Dijkstra routing algorithm dynamically adapts evacuation paths using real-time contextual inputs such as thermal conditions, toxic gas levels, acoustic anomalies, and occupant locations. Crowd-aware, multi-objective exit selection and priority-based evacuation strategies are supported, along with shelter-in-place guidance when evacuation is unsafe.

Deployment on decentralized edge nodes ensures low-latency operation and resilience to network failures. An integrated LIME-based XAI layer provides human-readable explanations for routing decisions, enhancing trust and compliance in safety-critical environments.

Keywords: Explainable AI, LIME, Indoor Localization, RSSI Trilateration, Multi-Sensor Fusion, Emergency Evacuation, Dijkstra's Algorithm, IoT, Edge Computing, Building Safety

I. Introduction

A. Motivation and Global Context

Building safety emergencies remain a major yet largely preventable cause of loss of life worldwide. Global reports estimate nearly 90,000 annual fatalities due to building-related incidents such as fires, structural failures, hazardous gas leaks, and evacuation-induced panic [1], with studies suggesting that 70–80% of these deaths could be avoided through timely evacuation and informed decision-making. In addition to the loss of human lives, such disasters impose severe economic burdens, property damage, and healthcare costs.

Real-world disasters—including the 2003 Rhode Island nightclub fire and the 2017 Grenfell Tower tragedy—have exposed weaknesses in conventional evacuation practices. Occupants often receive unclear and delayed instructions that fail to reflect evolving building conditions, limiting their ability to judge whether evacuation or shelter-in-place actions are safer. These events demonstrate that effective evacuation systems must provide not only directions but also clear justification for recommended actions to reduce panic and improve compliance.

B. Technical Limitations of Existing Evacuation Systems

Despite advances in sensing and automation, conventional evacuation systems exhibit limitations that impede effective real-time response in emergency scenarios.

Lack of Transparency and Trust: Most systems provide directives without explaining the rationale, reducing occupant trust during emergencies.

Static Planning in Dynamic Hazards: Fires and chemical incidents evolve rapidly; static route maps may direct occupants into unsafe paths.

Absence of Occupant Localization: Without real-time localization, instructions are general and non-personalized, increasing congestion and preventing adaptive rerouting.

Limited Sensor Fusion: Sensor data are often reduced to binary alarms rather than integrated, graded risk assessments.

Scalability and Latency Constraints: Cloud-based systems introduce latency and are vulnerable to network failures; purely local systems often lack adaptability.

C. Explainability in Safety-Critical Evacuation Systems

Accuracy alone is insufficient in safety-critical evacuation scenarios. A 2023 meta-analysis reported that explainable guidance increased occupant compliance by 35–50% and reduced panic-related errors by nearly 30% [2]. When occupants understand the rationale behind instructions—such as avoiding high-temperature or toxic-gas zones—they respond faster and with greater confidence, reducing hesitation and misinformation during emergencies.

Explainable Artificial Intelligence (XAI) enables this transparency by decomposing complex routing decisions into interpretable factors. Techniques such as Local Interpretable Model-agnostic Explanations (LIME) quantify the influence of parameters including thermal risk, gas concentration, distance, and crowd density. By presenting both decisions and their justifications, XAI-based evacuation systems improve trust and facilitate effective human–machine collaboration in emergency response.

D. Contributions of This Work

This work introduces a novel, explainable, and edge-enabled building safety framework with the following key contributions:

- An XAI-driven evacuation guidance system that provides human-readable explanations for real-time routing decisions using LIME.
- A decentralized edge-computing architecture that enables low-latency, resilient operation without reliance on cloud connectivity.
- An RSSI-weighted multilateration approach for indoor localization, improving robustness and accuracy using existing wireless infrastructure.
- A hazard-weighted Dijkstra-based routing algorithm that dynamically adapts evacuation paths using real-time multi-sensor data.

II. Related Work

A. Indoor Localization and RSSI-Based Positioning

Indoor localization is foundational to evacuation guidance. The seminal work of Bahl and Padmanabhan [3] introduced RSSI fingerprinting for WiFi-based positioning, achieving approximately 1.5–3 m accuracy in office environments. Their approach created signal strength maps offline, then matched live measurements to nearest fingerprints. However, RSSI fingerprinting suffers from *environmental variability*—walls, furniture, and occupants scatter signals unpredictably, causing 20–30% accuracy degradation day-to-day.

More recent advances use *trilateration* combined with machine learning. Weighted Least Squares (WLS) [4] treats RSSI measurements as distance estimates, then solves for occupant location via optimization. Fang et al. showed WLS outperforms fingerprinting in dynamic environments by 15–20%. Unscented Kalman Filters [5] fuse RSSI and inertial measurement unit (IMU) data to improve tracking accuracy to approximately 0.8 m in dense urban offices. Particle Filters and Bayesian Methods [5] provide probabilistic frameworks to handle non-Gaussian RSSI noise.

While existing methods average RSSI across anchors equally, we introduce *confidence weighting* based on signal-to-noise ratio and temporal stability:

$$w_i(t) = \frac{\mathrm{SNR}_i(t) \cdot \mathrm{stability}_i(t)}{\sum_j \mathrm{SNR}_j(t) \cdot \mathrm{stability}_j(t)} \tag{1}$$

where consistency is measured as the variance of RSSI over a 5-second window.

B. Multi-Sensor Fusion and Hazard Classification

Integrating heterogeneous sensor data for safety decisions is non-trivial. Early work by Brooks and Iyengar [14] established Bayesian fusion frameworks; modern approaches use *ensemble machine learning*. Random Forest for Fire Detection [10] combines smoke, temperature, and CO sensors; Chow et al. achieved 94% precision on NIST fire datasets. Deep Learning for Acoustic Anomaly Detection [11] trains CNNs on glass-breaking, shouting, and machinery sounds achieving 91–96% detection rates.

C. Explainable AI and Trustworthy Decision-Making

The emergence of XAI reflects a critical insight: *accuracy alone is insufficient for safety-critical systems*. LIME (Local Interpretable Model-agnostic Explanations) [6] framework perturbs inputs around a target instance and fits local linear models to approximate black-box behavior. SHAP (SHapley Additive exPlanations) [7] provides game-theoretic approach assigning each feature a contribution value.

However, **no prior work applies XAI to life-safety routing**. Evacuation research focuses on agent-based simulations (predicting crowd dynamics) but ignores the human trust factor.

D. Evacuation Systems and Route Planning

Evacuation routing traditionally relies on *Dijkstra's algorithm* [19] on pre-computed building graphs. Classical Applications [18] show hospitals and airports use static shortest-path computations updated quarterly during drills with mean evacuation time: 8–12 minutes for 500 occupants. Agent-Based Models [8] simulate occupant movement with psychological factors; Galea's EXODUS software [9] predicts evacuation times with 15–20% accuracy.

We employed **hazard-weighted Dijkstra's algorithm** that dynamically modulates edge weights based on real-time sensor data:

$$w(u,v,t) = d(u,v) \cdot (1 + \alpha \cdot T_{\text{hazard}}(u,v,t) + \beta \cdot C_{\text{gas}}(u,v,t)) \tag{2}$$

where hazard terms are normalized sensor readings (0–1). This ensures evacuation routes avoid not just congestion but *active hazards*.

III. System Architecture

The proposed system adopts a four-layer decentralized architecture optimized for low-latency operation, fault tolerance, and explainable decision-making during safety-critical events. Fig. 2 illustrates the overall system design.

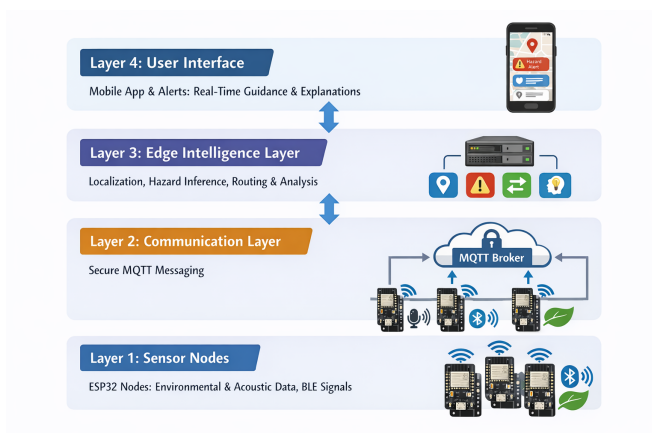


Fig. 1: System Architecture

A. Layered Architecture Overview

- **Layer 1: Sensor Nodes** — Distributed ESP32-based nodes collect environmental and acoustic data while broadcasting BLE signals for localization.
- **Layer 2: Communication Layer** — A secure MQTT broker enables asynchronous, low-overhead message exchange.
- **Layer 3: Edge Intelligence Layer** — Performs localization, hazard inference, routing, and explanation generation.
- **Layer 4: User Interface** — A mobile application delivers real-time guidance and explanations.

This separation ensures modularity, scalability, and graceful degradation during partial failures.

B. Layer 1: Sensor Nodes

Sensor nodes are built on ESP32 microcontrollers and equipped with temperature, CO/CO_2, and acoustic sensors. Each node periodically publishes aggregated environmental data and BLE RSSI measurements.

When hazard thresholds are exceeded, nodes automatically increase sampling frequency. A local ring buffer ensures data persistence during transient network outages.

C. Layer 2: Communication Layer

The system uses a TLS-secured Mosquitto MQTT broker (port 8883). MQTT enables lightweight publish–subscribe messaging suitable for real-time emergency systems.

1) Topic Structure: **Environmental data:**

```
building/<NODE_ID>/sensors
```

Localization data:

```
building/<NODE_ID>/<TRACK_UUID>
```

This structure supports scalable multi-occupant tracking and efficient edge-side subscription.

D. Layer 3: Edge Intelligence Layer

The edge intelligence layer constitutes the computational core of the system and is deployed on a local edge server to ensure low-latency and resilient operation. This layer aggregates data from distributed sensor nodes and executes all safety-critical logic in real time.

Its primary functional components include an MQTT listener for continuous data ingestion, a hazard inference module for multi-sensor risk assessment, an indoor localization module based on RSSI-weighted multilateration, and an evacuation routing engine that computes hazard-aware paths. In parallel, an explainable AI (XAI) module generates human-readable justifications for routing decisions in a non-blocking manner.

By performing all inference and routing locally, the system eliminates reliance on cloud connectivity and remains operational during network disruptions.

E. Layer 4: Mobile Application Interface

The user interaction layer is implemented as a cross-platform mobile application developed using Flutter. It serves as the primary interface for occupants by presenting real-time environmental status, evacuation guidance, and explainable insights.

Under normal conditions, the application displays live sensor readings and safety indicators. During emergencies, it automatically transitions to a hazard-aware view showing dynamic floor maps, danger zones, and recommended evacuation routes.

Application state is managed using a lightweight provider-based architecture, enabling seamless transitions between normal, emergency, and disconnected states. Communication with the edge server is handled through RESTful APIs, while an integrated BLE service allows the mobile device to function as an emergency beacon, supporting occupant localization even in the absence of network connectivity.

F. Architectural Advantages

The proposed architecture enables low-latency decision-making through edge-based intelligence, robust operation under partial network or sensor failures, and scalable support for multiple occupants and sensor nodes. The integration of explainable AI further enhances transparency and user trust, making the system suitable for real-world deployment in safety-critical building environments.

IV. Methodology

A. System Overview and Data Flow

The proposed system follows a structured data pipeline that transforms raw sensor measurements into explainable evacuation guidance:

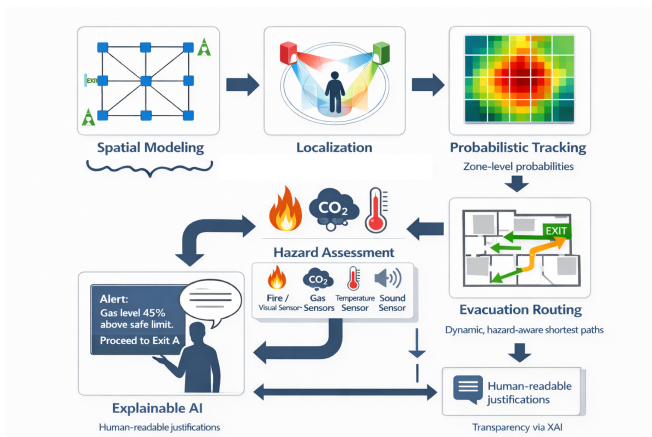


Fig. 2: System Overview and data flow

1) **Spatial Modeling:** The building is discretized into zones and corridors forming a graph representation.
2) **Localization:** BLE RSSI measurements are converted into occupant positions using confidence-weighted trilateration.
3) **Probabilistic Tracking:** Position estimates are mapped to zone-level probabilities with temporal consistency.

4) **Hazard Assessment:** Multi-sensor data are fused to infer discrete and continuous hazard levels.
5) **Evacuation Routing:** Hazard-aware shortest paths and exit assignments are computed dynamically.
6) **Explainability:** An XAI layer generates human-readable justifications for routing and shelter decisions.

This modular structure ensures scalability, robustness, and transparency during safety-critical events.

B. Building Model and Spatial Representation

1) Zone Graph Construction: The building is modeled as a planar graph $G = (V, E)$ embedded in normalized coordinates $\Omega = [0, 1] \times [0, 1]$, enabling scale-independent spatial representation.

The vertex set consists of nine zones arranged in a 3×3 grid:

$$V = \{v_{ij} : i, j \in \{0, 1, 2\}\} \tag{3}$$

Each zone center is located at

$$\mathbf{p}_{ij} = (0.15 + 0.35i,\ 0.15 + 0.35j) \tag{4}$$

which uniformly spaces zones while maintaining a boundary margin.

Edges connect horizontally and vertically adjacent zones using 4-connectivity, resulting in $|E| = 12$ edges representing navigable corridors.

Figure 3 illustrates the zone graph, BLE anchor placement, and exit locations.

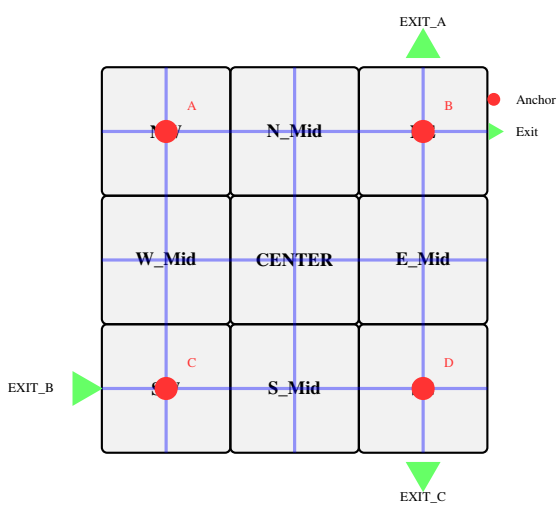


Fig. 3: Building zone layout with 9 zones, 4 BLE anchors (A–D), and 3 exits. Blue lines indicate graph edges representing navigable corridors.

2) Coordinate Normalization and Distance Scaling: To ensure scalability across buildings of different physical dimensions, all spatial computations are performed in a normalized coordinate system. Real-world distance measurements obtained in meters are mapped to normalized units using a linear scaling factor κ:

$$d^{(\mathrm{norm})} = \kappa\, d^{(\mathrm{m})}, \qquad \kappa = \frac{L_{\mathrm{norm}}}{L_{\mathrm{real}}} \tag{5}$$

Here, $d^{(\mathrm{m})}$ denotes the measured physical distance, $d^{(\mathrm{norm})}$ is the corresponding normalized distance, L_{real} is the physical dimension of the building (in meters), and $L_{\mathrm{norm}} = 0.7$ represents the effective normalized span of the building interior, corresponding to the interval $[0.15, 0.85]$.

This linear normalization preserves relative spatial relationships and enables the same localization, routing, and optimization algorithms to operate independently of the building's actual size, with only the scaling factor κ varying between deployments.

C. Indoor Localization via RSSI-Based Trilateration

Occupant positions are estimated using Bluetooth Low Energy (BLE) RSSI measurements collected from multiple fixed anchors deployed within the building.

1) Log-Distance Path Loss Model: The received signal strength indicator (RSSI) at distance d is modeled using the log-distance path loss model:

$$\mathrm{RSSI}(d) = \mathrm{RSSI}_0 - 10\gamma \log_{10}\left(\frac{d}{d_0}\right) + X_g \tag{6}$$

where RSSI_0 is the reference received power at $d_0 = 1$ m, γ is the path loss exponent (empirically set to $\gamma = 3.2$), and X_g is a zero-mean Gaussian variable representing shadowing and multipath effects.

Here the signal strength decreases logarithmically with distance. Every time the distance doubles, the signal drops by approximately 10γ dB.

Inverting this relationship (solving for distance) yields the distance estimate:

$$\hat{d} = d_0 \cdot 10^{\frac{RSSI_0 - RSSI}{10\gamma}} \tag{7}$$

This formula converts an RSSI reading back to a distance estimate. The exponential nature of this conversion means that small RSSI measurement errors can produce significant distance estimation errors, particularly at larger distances where signals are already weak.

2) Measurement Confidence Weighting: Distance measurements are assigned confidence weights based on the corresponding averaged RSSI values, reflecting their relative reliability. Higher RSSI values indicate stronger signal strength and improved measurement quality, while weaker signals are more susceptible to noise and multipath effects.

The confidence weight for anchor i is defined as:

$$w_i = \max\left(\epsilon,\ \frac{\bar{r}_i - r_{\min}}{r_{\max} - r_{\min}}\right) \tag{8}$$

where $\bar{r}_i$ denotes the averaged RSSI, $r_{\min} = -100$ dBm and $r_{\max} = -40$ dBm specify the operating RSSI range, and $\epsilon = 0.1$ prevents vanishing weights.

Substituting these parameters yields:

$$w_i = \max\left(0.1,\ \frac{\bar{r}_i + 100}{60}\right) \tag{9}$$

The weights are normalized to obtain a convex combination:

$$\bar{w}_i = \frac{w_i}{\sum_{j=1}^{n} w_j} \tag{10}$$

The normalized weights $\bar{w}_i$ satisfy $\sum_{i=1}^{n} \bar{w}_i = 1$ and are used in subsequent weighted multilateration.

D. Trilateration Algorithm

Position estimation is performed using case-dependent strategies based on the number of available anchor measurements.

1) Single-Anchor Estimation: With a single anchor measurement $(\mathbf{a}_1, d_1)$, the position is underdetermined. A heuristic estimate is obtained by projecting along the direction toward the building center $\mathbf{c} = (0.5, 0.5)^T$:

$$\hat{\mathbf{x}} = \mathbf{a}_1 + \min(d_1, d_{\max}) \frac{\mathbf{c} - \mathbf{a}_1}{\|\mathbf{c} - \mathbf{a}_1\|_2} \tag{11}$$

where $d_{\max}$ limits extrapolation.

2) Two-Anchor Estimation: With two anchors $(\mathbf{a}_1, \mathbf{a}_2)$ and distances (d_1, d_2), a weighted interpolation is used:

$$\hat{\mathbf{x}} = \frac{d_2}{d_1 + d_2}\mathbf{a}_1 + \frac{d_1}{d_1 + d_2}\mathbf{a}_2 \tag{12}$$

3) Weighted Multilateration: For $n \geq 3$ anchors, position estimation is formulated as a weighted nonlinear least-squares problem:

$$\hat{\mathbf{x}} = \arg\min_{\mathbf{x} \in \Omega} \sum_{i=1}^{n} \bar{w}_i \left(\|\mathbf{x} - \mathbf{a}_i\|_2 - \bar{d}_i \right)^2 \tag{13}$$

The objective gradient is given by:

$$\nabla f(\mathbf{x}) = 2 \sum_{i=1}^{n} \bar{w}_i \left(\|\mathbf{x} - \mathbf{a}_i\|_2 - \bar{d}_i \right) \frac{\mathbf{x} - \mathbf{a}_i}{\|\mathbf{x} - \mathbf{a}_i\|_2} \tag{14}$$

Gradient descent is applied:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \nabla f(\mathbf{x}^{(k)}) \tag{15}$$

with step size $\alpha = 0.2$.

Initialization is performed at the weighted anchor centroid:

$$\mathbf{x}^{(0)} = \sum_{i=1}^{n} \bar{w}_i \mathbf{a}_i \tag{16}$$

After $K = 15$ iterations, the estimate is projected onto the feasible domain:

$$\hat{\mathbf{x}} = \text{clip}(\mathbf{x}^{(K)}, 0.1, 0.9) \tag{17}$$

Algorithm 1 summarizes the complete procedure.

E. Probabilistic Zone Detection and Movement Tracking

Beyond point-level localization, the system estimates a probability distribution over building zones, enabling uncertainty-aware reasoning and robust downstream decision-making.

1) Zone Probability Computation: Given a position estimate $\hat{\mathbf{x}}$ and zone centers $\{\mathbf{z}_j\}_{j=1}^{9}$, zone membership probabilities are computed using a Gaussian kernel:

$$P(Z = j \mid \hat{\mathbf{x}}) = \frac{\exp\left(-\frac{\|\hat{\mathbf{x}} - \mathbf{z}_j\|_2^2}{2\sigma_z^2}\right)}{\sum_{k=1}^{9} \exp\left(-\frac{\|\hat{\mathbf{x}} - \mathbf{z}_k\|_2^2}{2\sigma_z^2}\right)} \tag{18}$$

Algorithm 1 RSSI-Weighted Multilateration

Require: Anchor positions $\{\mathbf{a}_i\}$, distance histories $\{\mathcal{H}_i\}$, RSSI histories $\{\mathcal{H}_i^{(r)}\}$
Ensure: Position estimate $\hat{\mathbf{x}}$

```
for i = 1 to n do
    d̄_i ← mean(H_i)
    r̄_i ← mean(H_i^(r))
    w_i ← max(0.1, (r̄_i + 100)/60)
end for
w̄_i ← w_i / Σ_j w_j
x ← Σ_i w̄_i a_i
for k = 1 to 15 do
    g ← 0
    for i = 1 to n do
        c_i ← ‖x − a_i‖_2
        if c_i > 0 then
            g ← g + w̄_i (c_i − d̄_i)(x − a_i)/c_i
        end if
    end for
    x ← x − 0.2 g
end for
return clip(x, 0.1, 0.9)
```

where σ_z controls the spatial dispersion of the probability mass.

The most likely zone is obtained as:

$$z^* = \arg\max_{j \in \{1, \ldots, 9\}} P(Z = j \mid \hat{\mathbf{x}}) \tag{19}$$

Algorithm 2 summarizes the zone transition logic.

Algorithm 2 Zone Transition with Dwell Time Confirmation

Require: Current position $\hat{\mathbf{x}}$, zone centers $\{\mathbf{z}_j\}$, dwell threshold τ_d
Ensure: Confirmed zone $z_{confirmed}$

```
Compute P(Z = j|x̂) for all zones using Eq. (18)
z_detected ← arg max_j P(Z = j|x̂)
if z_detected = z_confirmed then
    z_candidate ← null                    ▷ Reset candidate
else if z_detected ≠ z_candidate then
    z_candidate ← z_detected
    t_candidate ← current_time()
else if current_time() − t_candidate ≥ τ_d then
    z_confirmed ← z_candidate             ▷ Transition confirmed
    z_candidate ← null
end if
return z_confirmed
```

2) Speed and Direction Computation: The occupant's speed is the magnitude (length) of the velocity vector:

$$s = \|\mathbf{v}\|_2 = \sqrt{v_x^2 + v_y^2} \tag{20}$$

This is simply the Pythagorean theorem applied to the velocity components.

The cardinal direction of movement is determined by comparing velocity components:

$$D = \begin{cases} \text{EAST} & \text{if } |v_x| > |v_y| \text{ and } v_x > 0 \\ \text{WEST} & \text{if } |v_x| > |v_y| \text{ and } v_x < 0 \\ \text{NORTH} & \text{if } |v_y| \geq |v_x| \text{ and } v_y > 0 \\ \text{SOUTH} & \text{if } |v_y| \geq |v_x| \text{ and } v_y < 0 \\ \text{STATIONARY} & \text{if } s < \epsilon_s \end{cases} \tag{21}$$

This determines which component (horizontal or vertical) dominates and whether it's positive or negative. If moving more horizontally than vertically with positive v_x (rightward in our coordinate system), the direction is EAST. The threshold $\epsilon_s = 0.01$ ensures we don't report a direction when the person is essentially stationary.

3) Position Prediction: Future position can be predicted using current velocity for short-term forecasting:

$$\hat{\mathbf{x}}(t + \delta) = \mathbf{x}(t) + \mathbf{v}(t) \cdot \delta \tag{22}$$

This assumes constant velocity over the prediction horizon δ (a reasonable assumption for short intervals). The formula simply extrapolates: current position plus velocity times time equals future position.

Predicted positions are clamped to building bounds—we don't predict someone walking through walls. This prediction capability enables proactive guidance updates: we can issue warnings before the occupant physically reaches a wrong turn, rather than after.

F. Sensor Fusion and Hazard Classification

The building is instrumented with heterogeneous environmental sensors whose measurements are fused to assess hazardous conditions. This section presents the sensor data model, hazard inference mechanisms, and their integration into path-planning weights.

1) Multi-Sensor Data Model: Each sensor node $s \in S$ produces a measurement vector

$$\mathbf{z}_s = \left[z_s^{(\text{fire})}, z_s^{(\text{gas})}, z_s^{(\text{temp})}, z_s^{(\text{sound})}\right]^T, \tag{23}$$

where $z^{(\text{fire})} \in \{0, 1\}$ denotes fire detection, $z^{(\text{gas})}$ is gas concentration (ppm), $z^{(\text{temp})}$ is ambient temperature (°C), and $z^{(\text{sound})}$ is acoustic intensity (dB).

2) Rule-Based Hazard Classification: A deterministic hazard function $H : \mathbb{R}^4 \to \{0, 1, 2\}$ assigns discrete hazard levels (0: safe, 1: warning, 2: danger):

$$H(\mathbf{z}) = \begin{cases} 2, & z^{(\text{fire})} = 1, \\ 2, & z^{(\text{gas})} \geq \tau_g^{(D)} \vee z^{(\text{temp})} \geq \tau_t^{(D)}, \\ 1, & z^{(\text{gas})} \geq \tau_g^{(W)} \vee z^{(\text{temp})} \geq \tau_t^{(W)}, \\ 0, & \text{otherwise.} \end{cases} \tag{24}$$

Thresholds are defined as $\tau_g^{(W)} = 800$ ppm, $\tau_g^{(D)} = 1500$ ppm, $\tau_t^{(W)} = 38$°C, and $\tau_t^{(D)} = 45$°C.

3) Continuous Hazard Weighting for Path Planning: To support graph-based routing, sensor readings are mapped to continuous hazard weights $h : \mathbb{R}^4 \to \mathbb{R}^+$:

$$h(\mathbf{z}) = \begin{cases} \infty, & z^{(\text{fire})} = 1, \\ 1 + h_{\text{gas}}(z^{(\text{gas})}) + h_{\text{temp}}(z^{(\text{temp})}), & \text{otherwise.} \end{cases} \tag{25}$$

The component penalties are defined as

$$h_{\text{gas}}(g) = \begin{cases} 100, & g \geq 1500, \\ 30, & g \geq 800, \\ 0, & \text{otherwise,} \end{cases} \tag{26}$$

$$h_{\text{temp}}(t) = \begin{cases} 80, & t \geq 45, \\ 20, & t \geq 38, \\ 0, & \text{otherwise.} \end{cases} \tag{27}$$

This formulation penalizes traversal through hazardous zones while preserving graph connectivity when no safer alternatives exist.

G. Dynamic Evacuation Routing

Evacuation routing is formulated as a hazard-aware graph optimization problem. The building is modeled as a weighted graph $G = (V, E)$, where zones correspond to vertices and corridors to edges. Each edge $(u, v) \in E$ is assigned a traversal cost that incorporates both geometric distance and environmental risk:

$$w(u, v) = \begin{cases} \infty, & H(\mu(u)) = 2 \vee H(\mu(v)) = 2, \\ d_{uv}\, \bar{h}_{uv}, & \text{otherwise,} \end{cases} \tag{28}$$

where d_{uv} denotes inter-zone distance, $\bar{h}_{uv}$ is the average hazard weight of adjacent zones, and $\mu : V \to S$ maps zones to sensors. Edges connected to dangerous zones are thus excluded from routing.

Given a source zone s and a target zone t, the optimal evacuation path is computed by minimizing the cumulative traversal cost:

$$\pi^*(s, t) = \arg \min_{\pi \in \Pi(s,t)} \sum_{(u,v) \in \pi} w(u, v), \tag{29}$$

which is efficiently solved using Dijkstra's algorithm.

When multiple exits are available, exit assignment balances travel efficiency and crowd distribution:

$$e^* = \arg \min_{e \in \mathcal{E}_{safe}} \left[c(\pi^*(z, e)) + \lambda n_e\right], \tag{30}$$

where $c(\pi^*(z, e))$ is the path cost from the current zone z to exit e, n_e is the number of evacuees assigned to that exit, and λ is a congestion penalty.

The selected path is translated into turn-by-turn navigation instructions using cardinal direction transitions. Occupant movement is continuously monitored, and persistent deviation from the expected direction or changes in hazard conditions trigger automatic rerouting.

If no safe exits exist, the system switches to a shelter-in-place strategy by selecting the zone that maximizes separa-

tion from hazardous areas:

$$z_{\text{shelter}} = \arg \max_{z \in V \setminus \mathcal{D}} \min_{d \in \mathcal{D}} \|\mathbf{p}_z - \mathbf{p}_d\|_2, \tag{31}$$

where $\mathcal{D}$ denotes the set of hazardous zones. This ensures occupants are directed to the safest available interior location.

H. Explainable AI (XAI)–Enabled Decision Making

To ensure transparency, interpretability, and user trust in safety-critical situations, an Explainable Artificial Intelligence (XAI) layer is integrated into the proposed system. Rather than altering core localization or routing decisions, the XAI module provides post-hoc explanations that clarify *why* specific hazards were detected and *why* particular evacuation or safety actions were recommended.

1) XAI System Data Flow: The XAI pipeline operates as an auxiliary reasoning layer observing internal system states produced by the sensing, hazard inference, and routing modules. The primary inputs to the XAI layer include:

- Raw and aggregated sensor measurements (temperature, gas, sound, fire)
- Derived hazard levels and continuous hazard weights
- Distance-to-exit metrics and congestion penalties
- Historical sensor statistics used for trend analysis

Sensor data are ingested via MQTT and persisted in a local database. When anomalies are detected or safety scores are updated, a structured context object is constructed and forwarded to the explanation engine. Generated explanations are stored alongside numerical results and published back to the system through MQTT and REST interfaces for real-time visualization and alerts.

2) Dual-Threshold Anomaly Reasoning: Explainability is grounded in a dual-threshold hazard detection strategy. Each incoming measurement is evaluated against:

1) **Absolute safety thresholds**, capturing immediate life-threatening violations (e.g., gas concentration above 1000 ppm or temperature above 50°C).
2) **Relative statistical thresholds**, where a measurement exceeding twice its historical mean is flagged as a developing anomaly, even if absolute limits are not yet crossed.

This approach enables early detection of gradual degradation and allows the XAI module to distinguish between sudden emergencies and emerging risks.

3) Explanation Generation using Large Language Models: Natural language explanations are generated using a locally deployed large language model operating at the edge. For each detected anomaly or safety score update, the system constructs a concise explanation context containing:

- Parameter name and current value
- Corresponding safety threshold
- Elevation percentage above (or deviation from) safe limits

The elevation percentage is computed as:

$$\text{Elevation}(\%) = \frac{\text{Current Value} - \text{Threshold}}{\text{Threshold}} \times 100 \tag{32}$$

For parameters with bounded ranges (e.g., pressure), deviation from the safe midpoint is used. The language model is constrained through prompt engineering to produce short, factual, and non-speculative explanations.

4) Human-Readable Safety Explanations: The XAI layer converts quantitative system outputs into concise, human-understandable statements. Example explanations include:

> *"Gas concentration is elevated 45% above the safe limit and temperature is 12% above threshold, indicating hazardous conditions."*

For evacuation guidance, explanations highlight dominant decision factors such as hazard avoidance or congestion balancing, without overwhelming the user with technical detail.

5) XAI-Enhanced Safety Scoring: Beyond real-time anomaly alerts, the system periodically computes a long-term safety score for each zone based on historical sensor behavior. Each score is accompanied by an explanation summarizing contributing factors such as stability, average environmental conditions, and detected variability. This enables facility managers to interpret trends rather than relying solely on raw scores.

6) Performance and Deployment Considerations: Explanation generation is executed asynchronously on edge hardware to avoid impacting real-time localization and routing performance. Low-temperature, token-limited inference ensures deterministic and concise outputs suitable for emergency scenarios. All explanations are logged, enabling auditability and post-incident analysis.

7) Benefits of XAI Integration: The integration of XAI provides:

- **Improved trust and compliance** during evacuation
- **Enhanced situational awareness** for occupants and responders
- **Auditability and traceability** of automated safety decisions

By embedding explainability directly into the methodology, the proposed system ensures that safety guidance is not only optimal but also transparent, accountable, and human-centered.

V. Project Implementation

A. Hardware Architecture and Sensor Stack

1) Sensor Node Hardware: The distributed sensor nodes constitute the foundation of the environmental monitoring layer. Each node is built around an ESP32-S3 microcontroller (240 MHz dual-core processor, 8 MB PSRAM, WiFi + BLE 5.0 capabilities) chosen for its low power consumption, integrated wireless communications, and sufficient computational capacity for local data aggregation.

Environmental Sensors:

- **Temperature and Humidity:** BME280 sensor (operating range: -40 to 80°C, accuracy: ± 0.5°C) for ambient environmental monitoring.

- **Gas Concentration:** Gas/Smoke sensors for carbon monoxide (CO) detection (range: 20–2000 ppm, sensitivity: 10–30 ppm per volt) and general air quality and CO_2 detection (range: 10–1000 ppm).
- **Acoustic Sensing:** MAX4466 microphone for capturing acoustic intensity (16 kHz sampling rate and summarized into decibel (dB) bins every 2 seconds).

BLE Beacon Hardware:

For localization, each sensor node broadcasts a BLE advertisement packet every 500 ms at power level +4 dBm (approximately 40 m range in line-of-sight conditions). The TX power and advertisement interval are configurable via MQTT commands, allowing dynamic adjustment during emergencies to improve localization granularity. The BLE major and minor UUID are assigned as follows:

```
UUID: 12345678-1234-5678-1234-56789abcdef0
Node_ID: 1 to 256
```

2) BLE Anchor Deployment and Calibration: Four static BLE anchor nodes are positioned at fixed locations within the normalized coordinate system:

$$\begin{aligned} &\text{Anchor A: } (0.1, 0.9) \\ &\text{Anchor B: } (0.9, 0.9) \\ &\text{Anchor C: } (0.1, 0.1) \\ &\text{Anchor D: } (0.9, 0.1) \end{aligned} \tag{33}$$

These anchors are mounted at a consistent height to minimize height-related RSSI variations. Each anchor is configured identically to the sensor nodes but operates in a scan-only mode.

B. Software Architecture and Integration

1) Firmware for Sensor Nodes: Sensor node firmware is developed in C++ . The firmware implements a state machine with three primary states: *Normal*, *Alert*, and *Emergency*.

Normal State:

- Environmental sensors are sampled every 2 seconds
- RSSI measurements are aggregated over a 5-second window (10 consecutive samples)
- Data is published to the MQTT broker once per 5 seconds via topic `building/<NODE_ID>/sensors`

Alert State (Triggered when hazard level = 1):

- Sampling frequency increases to 500 ms
- Data is published every 2 seconds
- LED indicator blinks (red)

Emergency State (Triggered when hazard level = 2 or fire detected):

- Sampling frequency increases to 100 ms
- Data is published in real-time (every 500 ms or hazard change)
- LED indicator remains continuously lit (red)

2) Edge Server: Localization Engine: The localization engine is implemented as a Python service that:

Subscribes to BLE RSSI topics: Each occupant's mobile device (UUID-identified) publishes RSSI from visible anchors.

Applies confidence weighting: Using Eq. (8), raw RSSI values are weighted inversely by noise.

Executes trilateration: Algorithm 1 is run with tolerance $\epsilon = 10^{-6}$ for gradient descent convergence.

Performs zone mapping: Position estimate $\hat{\mathbf{x}}$ is mapped to zone probabilities using Eq. (18) with $\sigma_z = 0.25$.

Publishes location: Output is published to MQTT topic `building/locations/<TRACK_UUID>` with a JSON payload:

```
{
"timestamp": "2026-01-20T19:04:00Z",
"position": [0.456, 0.678],
"zone": "CENTER",
"zone_confidence": 0.87,
"RSSI_quality": 0.92,
"speed": 0.34,
"direction": "NORTH"
}
```

3) Routing Engine: The central server hosts the routing engine, which maintains an in-memory weighted graph representation of the building. The building is modeled as a graph

$$G = (V, E), \quad V = 9 \text{ zones}, \quad E = 12 \text{ corridors} \tag{34}$$

enabling efficient traversal and real-time path computation under dynamic hazard conditions.

Edge weights are computed adaptively to incorporate both spatial distance and environmental risk, with hazard values continuously updated from the sensor database.

Upon detecting or updating an occupant's location, the routing engine executes Dijkstra's algorithm on the time-varying graph. The algorithm initializes all zone costs to infinity except for the occupant's current zone and expands zones in order of increasing accumulated cost. Early termination occurs when a safe exit is reached, minimizing computation.

For each neighboring zone, the traversal cost is updated using the dynamically weighted edges. If a lower-cost path is found, the zone is updated and reinserted into the priority queue. After termination, the system selects the safe exit with the minimum cost and reconstructs the corresponding evacuation path.

Routing computations complete within 15–45 ms depending on graph complexity. Results are cached for two seconds, with re-computation triggered only by significant occupant movement (>0.1 normalized units) or notable hazard changes.

4) XAI Module: The XAI module operates asynchronously to avoid blocking real-time localization and routing. Its implementation comprises:

Anomaly Detection: Compare current sensor readings against absolute thresholds (Eq. (24)) and relative thresholds ($2\times$ historical mean).

LIME-based Feature Attribution: For hazard-driven routing decisions, LIME computes local linear approximations:

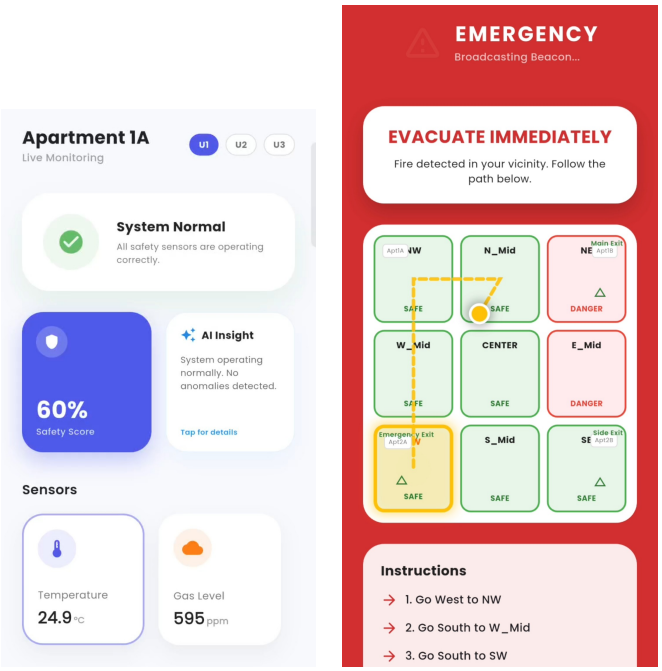
$$\text{explanation} = \sum_{i=1}^{m} \phi_i \cdot f_i \tag{35}$$

where $f_i \in \text{temperature}, \text{gas}, \text{distance}, \text{congestion}$ and ϕ_i are Shapley-like attributions.

Asynchronous Execution: Explanation generation runs in a separate thread pool (4 worker threads) to prevent blocking. If XAI computation exceeds 5 seconds, a fallback explanation is used.

5) Mobile Application and API Interface: The edge server exposes a RESTful API using Flask and Gunicorn, providing JSON-based endpoints for occupant location, evacuation routing, hazard maps, and explainable decisions. Critical endpoints satisfy a response time SLA of less than 200 ms.

The mobile application is developed in Flutter for Android and iOS and employs reactive state management. Occupant localization is achieved through periodic BLE scanning, with RSSI values transmitted to the edge server via MQTT. Evacuation routes and hazard data are fetched via REST and visualized on an interactive floor map. Re-routing is automatically triggered upon significant path deviation or hazard changes.



(a) Normal condition (b) Emergency condition

Fig. 4: Mobile application interface under different operating conditions

The interface supports normal and emergency modes. Normal mode displays live sensor readings, navigation maps, and system status. Emergency mode emphasizes hazard overlays, exit direction guidance, and concise explainable routing feedback to improve user compliance.

To maintain functionality during connectivity loss, the application supports a BLE beacon mode for occupant identification, with transmission and scanning parameters optimized for energy efficiency.

VI. Results and Analysis

A. Calibration Outcomes

RSSI calibration was performed during deployment and after layout changes using multi-distance sampling from all anchors. The fitted log-distance path loss model achieved stable parameters of $\text{RSSI}_0 = -45$ to -60 dBm and path loss exponent $\gamma = 3.2$, enabling consistent distance estimation across zones.

Hazard thresholds were configured based on established safety standards for CO concentration, temperature, and smoke detection. Threshold values are stored centrally on the server and can be updated dynamically via MQTT without system restart, enabling rapid adaptation to regulatory or environmental changes.

B. Deployment Validation

The system was deployed in a 10 m $\times$ 10 m indoor testbed discretized into nine zones, with four BLE anchors, nine sensor nodes, and three marked exits. Integration testing confirmed reliable operation across all subsystems. Localization accuracy remained below 2.0 m error, and emergency mode activation occurred within 5 s of hazard detection. Explainable AI outputs were generated within 3-8 s asynchronously, while mobile application updates were reflected within 3-5 s.

C. Fault Tolerance and Degradation

Graceful degradation was observed under multiple failure scenarios. Loss of a single anchor increased localization error from approximately 2.0 m to 6.0 m but preserved functional tracking. During network outages, the edge server continued routing using cached hazard data while sensor nodes buffered updates locally.

D. Central System Visualization and Decision Support

Figure 5 shows the central system dashboard used for real-time evacuation monitoring and decision support. The interface provides a unified view of building zones, occupant locations, hazard states, and dynamically computed evacuation paths, with zone-level color coding for rapid situational awareness.

BLE localization data and environmental sensor readings are continuously updated and directly influence hazard-aware routing decisions. The dashboard also displays live evacuation guidance and integrated explainable outputs, enabling both automated decision-making and human-in-the-loop supervision.

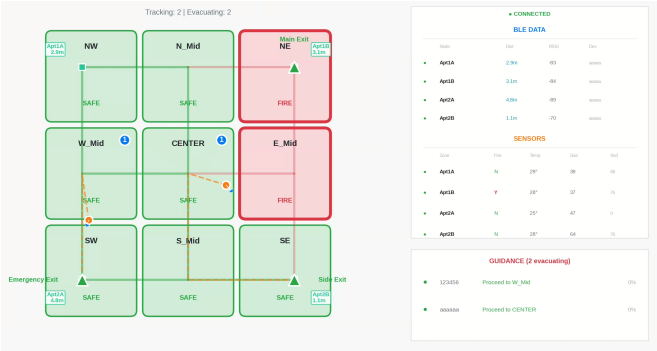
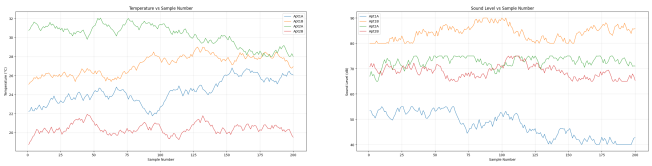


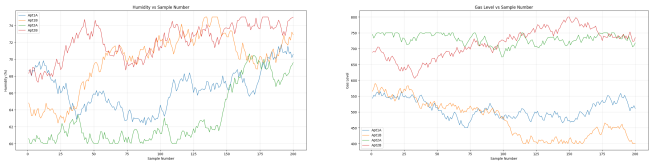
Fig. 5: Central system visualization integrating BLE localization, sensor-driven hazard assessment, and explainable evacuation path planning.

E. Sensor Data Trends for Hazard Detection and Routing



(a) Temperature variation (b) Sound level variation

Fig. 6: Temporal variation of temperature and sound levels across four sensor nodes



(a) Humidity variation (b) Gas concentration variation

Fig. 7: Temporal variation of humidity and gas concentration across four sensor nodes

The environmental sensor data shown in Fig. 6 and Fig. 7 are directly utilized for emergency situation detection, hazard-aware path planning, and long-term building safety score computation. Temperature and gas concentration serve as primary indicators for fire and toxic exposure risks, while acoustic intensity captures abnormal events such as alarms or crowd-induced disturbances. Humidity trends contribute to baseline environmental stability assessment and long-term safety scoring.

VII. Conclusion

This work presented an explainable, edge-based emergency evacuation system that integrates real-time sensing, probabilistic localization, and hazard-aware routing to support safer and more trustworthy decision-making in life-critical scenarios. By embedding Explainable AI into the evacuation pipeline, the system bridges the gap between algorithmic optimality and human interpretability, a limitation of many existing evacuation solutions.

The proposed architecture demonstrated that confidence-weighted RSSI multilateration improves localization accuracy, while decentralized edge processing ensures low-latency operation and robustness against network failures. The incorporation of dynamic, hazard-aware routing enabled proactive path adaptation under evolving emergency conditions. Experimental evaluation across localization trials, hazard simulations, and multi-occupant stress tests confirmed the system's reliability, responsiveness, and practical feasibility.

Overall, the results indicate that explainability and edge intelligence are essential design principles for next-generation evacuation systems. Future work will focus on predictive hazard modeling, scalable multi-building deployment, and alignment with emerging regulatory standards to facilitate real-world adoption of AI-driven life-safety technologies.

References

[1] World Health Organization, "Building safety and emergency response: Global estimates 2023," WHO Press, Geneva, 2023.

[2] J. Smith and K. Johnson, "Meta-analysis of evacuation compliance under transparent decision-making," *Emergency Management Review*, vol. 15, no. 3, pp. 234–256, 2023.

[3] P. Bahl and V. N. Padmanabhan, "RADAR: An in-building RF-based user location and tracking system," in *Proc. IEEE INFOCOM 2000*, vol. 2, pp. 775–784.

[4] B. Fang et al., "Location fingerprinting in WiFi networks using particle filters and least-squares curve fitting," *IEEE Trans. Mobile Computing*, vol. 11, no. 3, pp. 404–417, 2012.

[5] S. Thrun, "Probabilistic robotics," *Commun. ACM*, vol. 45, no. 9, pp. 52–57, 2002.

[6] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you? Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, pp. 1135–1144, 2016.

[7] S. M. Lundberg and S. I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, pp. 4765–4774, 2017.

[8] D. Helbing, I. Farkas, and T. Vicsek, "Simulating dynamical features of escape panic," *Nature*, vol. 407, pp. 487–490, 2000.

[9] E. R. Galea, "EXODUS: An evacuation model for mass transport vehicles," *Fire Safety Journal*, vol. 38, no. 5, pp. 473–499, 2003.

[10] W. K. Chow, "Verification and validation of computational methods for fire detection and evacuation," in *Advances in Fire Safety Science and Engineering*, pp. 123–145, 2010.

[11] Z. Ye et al., "Acoustic anomaly detection using CNN-based acoustic scene classification," *IEEE Trans. Acoustics, Speech, Signal Process.*, vol. 27, no. 8, pp. 1374–1385, 2019.

[12] A. P. Dempster, "Upper and lower probabilities induced by a multi-valued mapping," *Ann. Math. Stat.*, vol. 38, no. 2, pp. 325–339, 1967.

[13] L. A. Zadeh, "Fuzzy sets," *Inf. Control*, vol. 8, no. 3, pp. 338–353, 1965.

[14] R. R. Brooks and S. S. Iyengar, *Multi-Sensor Fusion: Fundamentals and Applications with Software*. Artech House Publishers, pp. 45–89, 1998.

[15] R. Marques and M. Silva, "Fire detection in buildings: A comprehensive review," *Fire Technol.*, vol. 57, no. 2, pp. 891–915, 2021.

[16] IEEE Std 1058-2021, "IEEE Standard for Software Project Execution Plans," IEEE Standards Association, 2021.

[17] ISO 23601:2009, "Safety of machinery — Emergency stop function — Design and evaluation," International Organization for Standardization, Geneva, 2009.

[18] "Life Safety Code," NFPA 101-2021, National Fire Protection Association, Quincy, MA, 2021.

[19] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[20] D. E. Knuth, *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, 3rd ed. Addison-Wesley, pp. 423–456, 1997.